# Google Meet & Chat Enterprise Collaboration Design & Configuration

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

# Contents


Executive Summary 4

Solution Overview 4

- What this is . . . 4
- How it works . . . 5
- Reference model . . . 5
- Benefits . . . 6
- Limitations & boundaries . . . 6
- Deployment model . . . 7

Requirements 7

- 1. Workspace edition that includes the required Meet, Chat, and Vault controls . . . 7
- 2. OU and configuration-group structure for policy targeting . . . 8
- 3. A governed Shared Drive for recordings, transcripts, and meeting notes . . . 8
- 4. Documented retention and eDiscovery policy for Meet and Chat data . . . 9
- 5. External-collaboration policy decision (Meet join & external Chat) . . . 9
- 6. Chat apps/bots inventory and an approved allowlist . . . 9
- 7. API access for the automation tools (least-privilege service identity) . . . 9

Global Requirements — Google Workspace & Gemini Solutions 10

- 1. Google Workspace / Cloud Identity tenant . . . 10
- 2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 10
- 3. DNS control for the routed domains . . . 11
- 4. Identity & provisioning source of truth . . . 11
- 5. A pilot scope (OU / group) and change control . . . 11
- 6. Security, compliance & governance stakeholders . . . 12

Implementation 12

- Phase 0 — Assess (read-only) . . . 12
- Phase 1 — Design . . . 13
- Phase 2 — Configure (Pilot OU/group first) . . . 13
- Phase 3 — Pilot . . . 15
- Phase 4 — Validate . . . 15
- Phase 5 — Expand . . . 15

Testing & Validation 16

- Test 1 — Meet safety & host controls enforce (`Meet Safety Policy – Pilot OU – Configure`) . . . 16
- Test 2 — External Meet access is scoped (`Meet External Access – Partners Group – Scoped`) . 16
- Test 3 — Recordings land in the governed Shared Drive (`Recording Storage – Meet Recordings Shared Drive – Configure`) . . . 17

Test 4 — Chat external policy matches the allowlist (`Chat External Policy – Allowlisted Domains – Enforce`) . . . 17
Test 5 — File sharing in Chat follows policy . . . 17
Test 6 — Chat history & Vault retention apply (`Chat History – Default On – Enforce`, `Vault Retention – Chat 3yr – Enforce`) . . . 17
Test 7 — Spaces taxonomy & creation scope (`Chat Spaces Taxonomy – Enterprise – Design`) . 18
Test 8 — App/bot allowlist enforces (`Chat App Allowlist – Approved Bots – Enforce`) . . . 18
Test 9 — Automated settings & health validation . . . 18
Troubleshooting 18
Configuration & scoping . . . 19
Recording & storage . . . 19
External access & retention . . . 20
Automation / API . . . 20
Best Practices 21
Scope by OU and group deliberately . . . 21
Keep external collaboration deliberate . . . 21
Govern recordings and transcripts centrally . . . 22
Make Chat eDiscovery-ready . . . 22
Run a real spaces taxonomy . . . 22
Contain third-party app risk . . . 22
Operate the controls as living policy . . . 22
Keep automation safe and auditable . . . 23
Deliverables Summary 23
Documentation . . . 23
The 5 Python tools (Google Workspace Admin SDK / Reports API) . . . 24
Appendix A — PowerShell Scripts 25

# Executive Summary

**Outcome.** Google Meet and Google Chat are configured for **productive, well-governed collaboration** across the enterprise: meetings are safe and host-controlled, recordings and transcripts land in one governed location with defined retention, external access is scoped to approved partners and domains, Chat spaces follow a real taxonomy with owners, message history and Vault retention make Chat eDiscovery-ready, and only approved apps and bots can be installed.

**The problem.** Meet and Chat are powerful by default but insecure by default. Rolled out unmanaged, they leave gaps: meetings anyone can join or disrupt, recordings scattered across personal drives, external meetings and external chat wide open, ad-hoc spaces with no ownership, no retention posture, and any user able to add third-party bots. Those gaps are collaboration-productivity risks *and* compliance risks.

**The approach.** The engagement designs a named, OU/group-scoped **Meet + Chat collaboration model** and configures it through the Google Admin console and Google Vault following the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. Every control — Meet safety/host, recording storage, external rules, Chat spaces, external/file policy, history/retention, app allowlist — is decided, named, applied to a **Pilot OU/group first**, validated against explicit pass/fail gates, and only then rolled outward wave by wave. Five Python tools built on the Google Workspace Admin SDK and Reports API assess the current state, help apply and expand the configuration (dry-run by default), validate external/retention posture, health-check for drift, and report usage — all read-only or dry-run unless an operator explicitly applies a change, with no credentials in code.

**Why it is low-risk.** Nothing is changed fleet-wide before it is proven on a pilot and passes the validation gates. High-impact changes (external access, history/retention, recording storage) are staged under change control, each with a documented rollback to a captured baseline. The tools default to read-only/dry-run and use least-privilege credentials. The result is collaboration that is both easy for users and defensible for security, records, and legal — delivered self-guided at a fraction of a consulting engagement's cost.

# Solution Overview

## What this is

This solution designs and configures the **steady-state collaboration model for Google Meet and Google Chat** across an enterprise Google Workspace tenant. It is deliberately distinct from a Teams-to-Workspace *migration*: here Meet and Chat are already available, but they were rolled out on defaults. That leaves gaps — meetings anyone can join or take over, recordings landing in uncontrolled Drive locations, external participants and external chat wide open, ad-hoc Chat spaces with no naming or ownership discipline, no message history/retention posture, and third-party Chat apps and bots installable by anyone.

The engagement produces a **named, OU/group-scoped policy design** for both services and configures it pilot-first:

- **Meet safety & host controls** — host management, who can join, quick access, moderation

(mute/remove, present locks), safety defaults for the organization and per OU/group.

• **Recording, transcription & storage policy** — who can record, where recordings/transcripts/notes land (a governed Shared Drive), and how they are retained.

• **External-participant rules for Meet** — whether and how people outside the organization can join, and warning/trust banners for external attendees.

• **Chat spaces architecture** — a spaces taxonomy (team, project, cross-functional, external), naming and ownership conventions, and who can create spaces.

• **External chat & file-sharing policy** — external chat allow/block, allowlisted domains, and file-sharing controls inside Chat.

• **History & retention** — Chat history default (on/off, per space), Meet artifact retention, and the Vault retention/hold rules that back eDiscovery.

• **App & bot governance** — which Chat apps/bots are allowed, and restriction of third-party app installation.

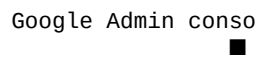
## How it works

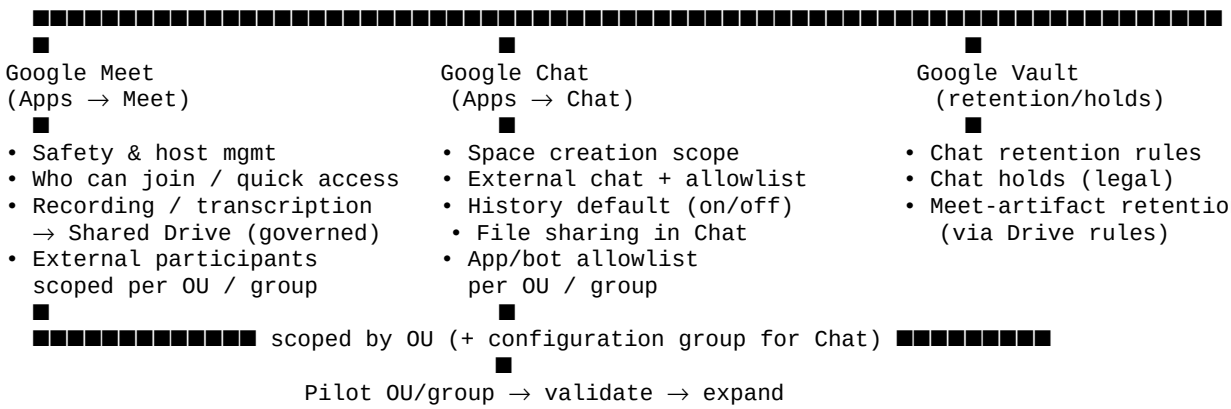
All policy is expressed in the **Google Admin console** (admin.google.com) under **Apps** → **Google Workspace** → **Google Meet** and → **Google Chat**, scoped by **organizational unit (OU)** and, for Chat, by **configuration group** where per-group targeting is needed. Retention and holds are set in **Google Vault** (vault.google.com). Recordings, transcripts, and Gemini-generated meeting notes are directed to a governed **Shared Drive** so their access and retention are controlled centrally rather than scattered across users' My Drive.

Every artifact this solution creates — a policy scope, a Shared Drive, a retention rule, a Chat app allowlist entry, a validation report — is **named** with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in configuration, testing, troubleshooting, and the Python tools. Examples used throughout:

- `Meet Safety Policy – Pilot OU – Configure`
- `Meet Host Management – All Staff – Enforce`
- `Recording Storage – Meet Recordings Shared Drive – Configure`
- `Meet External Access – Partners Group – Scoped`
- `Chat Spaces Taxonomy – Enterprise – Design`
- `Chat External Policy – Allowlisted Domains – Enforce`
- `Chat History – Default On – Enforce`
- `Vault Retention – Chat 3yr – Enforce`
- `Chat App Allowlist – Approved Bots – Enforce`

## Reference model






**Scoping model.** Meet and Chat settings apply **top-down by OU**, with child OUs inheriting unless overridden. Chat additionally supports **configuration groups** for targeting a policy at a group of users who may span OUs (for example, an "External Collaboration – Partners" group that is allowed external chat while the rest of the organization is not). The design assigns each control to the narrowest scope that satisfies policy.

## Benefits

• **Meetings are safe by default** — hosts are in control, uninvited join is prevented, and external attendees are visibly flagged.

• **Recordings and transcripts are governed** — they land in one controlled Shared Drive with known access and retention, not scattered in personal drives.

• **External collaboration is deliberate, not accidental** — external Meet join and external Chat are allowed only where the business needs them, to named domains.

• **Chat is organized and accountable** — a spaces taxonomy with owners and naming, not sprawl.

• **Compliance is satisfiable** — history and Vault retention/holds make Chat eDiscovery-ready.

• **Third-party risk is contained** — only approved Chat apps/bots can be installed.

## Limitations & boundaries

• This solution covers **Meet and Chat configuration and governance only**. It is not the Teams→Chat/Meet *migration* (that is a separate solution) and does not migrate historical Teams messages.

• **Admin-console labels and the availability of individual settings vary by Workspace edition and change over time** — treat exact toggle names as current-at-authoring and *verify in your tenant*. Vault, DLP, and some safety controls require the appropriate edition (e.g. Enterprise/Education Plus); confirm licensing (global requirement #1).

• **Gemini-in-Meet features** (automatic note-taking / "take notes for me", transcription behaviors) are edition- and rollout-dependent — *verify in your tenant* before relying on them.

• The Python tools use the **Google Workspace Admin SDK and Reports API**. Some Meet/Chat policy

surfaces are managed only in the Admin console UI and are **not** exposed as writable API settings; where that is so, the tools **assess and report** (read-only) and the console remains the change surface. No tool invents an API that does not exist.

### Deployment model

The engagement follows the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first, with explicit pass/fail gates and a documented rollback for every production change:

1. **Assess** — inventory current Meet/Chat settings, external exposure, spaces, retention, and installed apps (read-only tools + Admin console).
1. **Design** — decide the target policy per OU/group: Meet safety/host, recording storage, external rules, Chat spaces taxonomy, external/file policy, history/retention, app allowlist.
1. **Configure** — apply the design to the **Pilot OU/group** first, naming every artifact.
2. **Pilot** — a small representative user set runs Meet/Chat under the new policy.
3. **Validate** — run the test procedures and gates; confirm controls enforce, external is scoped, recordings land in the Shared Drive, retention applies.
1. **Expand** — roll the validated policy outward OU by OU, re-validating at each wave, with rollback ready.

High-impact changes (external access, history/retention, recording storage) are staged under change control; each has a named rollback in the Implementation guide.

## Requirements

These are the **solution-specific** prerequisites for designing and configuring Google Meet and Google Chat. They build on the shared **Global Requirements — Google Workspace & Gemini Solutions** (`global-requirements-google.md`): a provisioned Workspace/Cloud Identity tenant with a super-admin and appropriate edition (#1), a defined identity source of truth and provisioning (#4), a **pilot OU/group and change control** (#5), and engagement with **security/legal/data-governance stakeholders** (#6). Verify those first; the items below are what *this* solution additionally needs. (Requirements #2 M365/Entra and #3 DNS from the global set do not apply here — this is steady-state Meet/Chat configuration, not a migration or mail-flow change.)

Each requirement is stated as **Description / Why required / How to verify / How to implement if missing.**

### 1. Workspace edition that includes the required Meet, Chat, and Vault controls

• **Description:** A Workspace edition whose feature set includes the Meet safety/host controls, recording/transcription, Chat history and space-management controls, and **Google Vault** retention/ holds that the design relies on (generally a Business Plus / Enterprise Standard-or-Plus / Education Plus tier —

*verify the exact matrix for your edition*).

• **Why required:** Recording, transcription, advanced host management, and Vault retention/holds are not present in every edition. Designing a control the edition cannot enforce fails validation.

• **How to verify:** In the Admin console, **Billing** → **Subscriptions**, confirm the edition and seat counts; cross-check that **Google Vault** appears under **Apps** and that Meet **recording** and **Chat history** controls are visible under each app's settings.

• **How to implement if missing:** Upgrade the edition or add the required add-on through Google or an authorized reseller before configuring the dependent controls; until then, scope the design to the controls the edition supports and flag the rest.

## 2. OU and configuration-group structure for policy targeting

• **Description:** An organizational-unit tree plus a small set of **configuration groups** that model how Meet/Chat policy should differ across the organization — at minimum a **Pilot OU**, an all-staff baseline, and groups for special cases (e.g. `External Collaboration – Partners` for users allowed external chat/Meet, `Meet Recorders – Approved` for who may record).

• **Why required:** Meet settings apply by OU; Chat supports OU **and** configuration-group targeting. Without the right scopes you cannot apply a permissive external policy to a partner-facing team while keeping the rest of the organization locked down.

• **How to verify:** In **Directory** → **Organizational units**, confirm the OU tree; in **Directory → Groups**, confirm the configuration groups exist and membership is accurate.

• **How to implement if missing:** Create the Pilot OU/group (global requirement #5) and the special-case groups; agree membership ownership with the business before applying policy.

## 3. A governed Shared Drive for recordings, transcripts, and meeting notes

• **Description:** A dedicated **Shared Drive** — named `Recording Storage – Meet Recordings Shared Drive – Configure` — with defined membership, access levels, and sharing restrictions, to receive Meet recordings, transcripts, and Gemini meeting notes.

• **Why required:** By default recordings save to the organizer's My Drive, scattering sensitive artifacts with inconsistent access and no central retention. A governed Shared Drive gives one controlled location where access and Vault/Drive retention apply uniformly.

• **How to verify:** In **Apps** → **Google Drive and Docs** → **Manage Shared Drives**, confirm the Shared Drive exists with the intended members and sharing settings.

• **How to implement if missing:** Create the Shared Drive, set membership (managers/legal/records as content managers, meeting owners as contributors as policy dictates), restrict external sharing, and record its ID for use in the recording-storage configuration and the Python tools.

## 4. Documented retention and eDiscovery policy for Meet and Chat data

• **Description:** A written **retention schedule** and **legal-hold process** owned by legal/records governance that states how long Chat messages and Meet recordings/transcripts must be kept, per space type or OU, and how holds are placed.

• **Why required:** Chat history and Vault retention are governance decisions, not technical defaults; turning history off where the record must be kept, or retaining forever where policy says delete, both create risk.

• **How to verify:** Obtain the retention schedule and confirm the named approver (ties to global requirement #6).

• **How to implement if missing:** Work with legal/records to define retention periods per data type and the hold procedure before configuring Vault rules (`Vault Retention – Chat 3yr – Enforce` and peers are placeholders until the real periods are agreed).

## 5. External-collaboration policy decision (Meet join & external Chat)

• **Description:** A business decision, approved by security, on **whether external participants may join Meet**, whether external (cross-organization) Chat is allowed**, and if so, to** which domains** — captured as an allowlist for `Chat External Policy – Allowlisted Domains – Enforce`.

• **Why required:** External access is the highest-risk collaboration setting. The design must apply a deliberate, approved posture (open to named domains vs. blocked) rather than the permissive default.

• **How to verify:** Confirm a signed-off external-collaboration statement and the domain allowlist.

• **How to implement if missing:** Facilitate the decision with security/business owners; default to the restrictive posture (external blocked, warning banners on) until an allowlist is approved.

## 6. Chat apps/bots inventory and an approved allowlist

• **Description:** An inventory of Chat apps/bots currently in use and an **approved allowlist** of which third-party apps may be installed — `Chat App Allowlist – Approved Bots – Enforce`.

• **Why required:** Unrestricted app installation lets any user add a third-party bot with access to space content; governance requires an allowlist and a request path for new apps.

• **How to verify:** Use the assessment tool / Admin console to list installed Chat apps; confirm security has reviewed and approved the allowlist.

• **How to implement if missing:** Produce the inventory (assessment tool below), have security approve the allowlist, and define how users request additions before enforcing app restrictions.

## 7. API access for the automation tools (least-privilege service identity)

• **Description:** A way to run the Python tools against the Admin SDK and Reports API — either an

admin running with **OAuth** and read-only admin scopes, or a **service account with domain-wide delegation** limited to the read-only Directory/Reports scopes the tools declare, plus (only where a tool applies a change) the specific write scope, gated behind `--apply`.

- **Why required:** The assessment, validation, health, and reporting tools call

`admin.googleapis.com` (Directory + Reports). Least-privilege credentials keep the automation safe and auditable; no keys are embedded in code.

- **How to verify:** Confirm the OAuth client or service account exists, the required scopes are

authorized in **Security** → **API controls** → **Domain-wide delegation** (for a service account), and a read-only run of the assessment tool succeeds.

- **How to implement if missing:** Create the OAuth client / service account, authorize **only** the

read-only scopes for assessment/validation, and add a write scope only for the configuration helper under change control. Store credentials via ADC / a secured secret store — never in the repo.

# Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

## 1. Google Workspace / Cloud Identity tenant

- **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin**

account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

- **Why required:** All configuration, policy, and API work is performed against this tenant.

- **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license

counts under **Billing / Subscriptions**.

- **How to implement if missing:** Provision the tenant and licenses through Google or an authorized

reseller before starting.

## 2. Source Microsoft 365 / Entra environment (for migration & coexistence)

- **Description:** For migration/coexistence/identity solutions, administrative access to the source

**Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

## 3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

## 4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

## 5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the

owners before making production changes.

### 6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

## Implementation

Follow the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, pilot-first. Configure everything against the **Pilot OU/group** first, validate against the gates, then expand OU by OU. Every artifact is **named** on first creation and the exact name is reused later. Admin-console labels are current-at-authoring — **verify in your tenant**, as edition and UI wording change.

> ***High-impact changes*** *(external access, Chat history/retention, recording storage) are staged under change control. Each phase below states its* ***pass/fail gate*** *and a* ***rollback****.*

### Phase 0 — Assess (read-only)

1. Sign in to **admin.google.com** as super-admin.
2. Run the assessment tool to capture the current state into a baseline report:

```
python meet_chat_settings_assessment.py --domain your-domain.com --out baseline.json
```

This records Meet and Chat settings visible via the Admin SDK, the OU/group structure, and installed Chat apps. Name the artifact `Meet+Chat Baseline – Enterprise – Assess`.

1. Run the usage/adoption report (Reports API, read-only) to size current Meet/Chat activity and external exposure:

```
python meet_chat_usage_report.py --domain your-domain.com --days 30 --out usage.csv
```

1. In the Admin console, manually confirm settings the API does not expose (some safety and space-management toggles are UI-only). Record them in the baseline.

**Gate 0 (pass/fail):** PASS when the baseline documents current Meet safety, recording location, external Meet/Chat posture, spaces creation scope, history/retention, and installed apps for the whole tenant. FAIL if any of these is unknown — do not configure blind.

## Phase 1 — Design

Decide the target policy and record it in a design register (the design doc + the config helper's input file). For each control, choose the **narrowest scope** that satisfies policy:

• **Meet safety/host:** host management on; who can join (organization-only vs. allow external);

quick access; moderation defaults. Artifacts: `Meet Safety Policy – Pilot OU – Configure`, `Meet Host Management – All Staff – Enforce`.

• **Recording/transcription storage:** who may record (`Meet Recorders – Approved` group); recordings/

transcripts/notes to `Recording Storage – Meet Recordings Shared Drive – Configure`.

• **External Meet:** `Meet External Access – Partners Group – Scoped` (allowed for the partner group,

blocked elsewhere); external-participant warning on.

• **Chat spaces taxonomy:** `Chat Spaces Taxonomy – Enterprise – Design` — space types, naming

convention `<type>-<owner>-<purpose>`, ownership, and who can create spaces.

• **External/file Chat policy:** `Chat External Policy – Allowlisted Domains – Enforce`; file-sharing

in Chat controlled per policy.

• **History/retention:** `Chat History – Default On – Enforce`; `Vault Retention – Chat 3yr – Enforce`

(use real periods from Requirement #4); Meet-artifact retention via Drive/Vault rules on the Shared Drive.

• **App/bot governance:** `Chat App Allowlist – Approved Bots – Enforce`.

**Gate 1:** PASS when every control has a named target value, a scope (OU or group), and an approver where governance requires one (external policy, retention, recording storage). FAIL if any control is undecided.

## Phase 2 — Configure (Pilot OU/group first)

### *2a. Meet safety & host controls*

1. **Apps** → **Google Workspace** → **Google Meet** → **Safety settings.** Scope the console to the **Pilot OU**.

2. Set **Host management** default **on** so hosts control moderation — artifact `Meet Host Management

– All Staff – Enforce`.

1. Set **Who can join meetings created by your organization** and **Warn for external participants** to

the designed values; enable moderation defaults (mute, present locks). This is `Meet Safety Policy – Pilot OU – Configure`.

1. Save; confirm the OU shows the overridden values (not "Inherited").

### *2b. Recording / transcription storage*

1. Ensure the Shared Drive `Recording Storage – Meet Recordings Shared Drive – Configure` exists with

restricted external sharing (Requirement #3).

1. **Google Meet** → **Meet video settings** → **Recording.** Enable recording only for the **Pilot OU** and

restrict who can record to the `Meet Recorders – Approved` group.

1. Configure recordings/transcripts (and Gemini notes where the edition provides them — *verify in your

tenant*) to save to the governed Shared Drive per the current Admin-console option for recording storage location. Where the console does not expose a storage-location control, enforce it operationally (organizers record into the Shared Drive) and validate location in Phase 4.

### *2c. External participant rules (Meet)*

1. For the general population OU, set **Meet external access** to the restrictive posture (organization

participants only or external with warning), per the approved decision (Requirement #5).

1. For the `Meet External Access – Partners Group – Scoped` group (via a group override or a partner

OU), allow external join. Keep the external-participant **warning banner** on everywhere.

### *2d. Chat spaces architecture & creation scope*

1. **Apps** → **Google Workspace** → **Google Chat** → **Space creation.** Scope to the Pilot OU.

2. Apply `Chat Spaces Taxonomy – Enterprise – Design`: publish the naming convention and, if the

edition supports it, restrict who can create spaces / restrict space types per policy.

1. Assign owners for baseline spaces; document ownership in the taxonomy register.

### *2e. External chat & file-sharing policy*

1. **Google Chat** → **External chat settings.** For the general OU, set external chat to **blocked** or

**allowlisted-domains-only** per the approved posture.

1. Add the approved domains to `Chat External Policy – Allowlisted Domains – Enforce`.

2. Grant permissive external chat to the `External Collaboration – Partners` configuration group where

the business needs it. Configure **file sharing in Chat** to the designed value.

### *2f. History & retention*

1. **Google Chat** → **Chat history.** Set the default per policy — `Chat History – Default On – Enforce`

(or off where policy requires), and whether users can change it per space.

1. In **Google Vault** → **Retention** → **Google Chat**, create the retention rule `Vault Retention – Chat 3yr

– Enforce` (use the real period from Requirement #4), scoped to the Pilot OU first.

1. Apply Meet-artifact retention to the Shared Drive (Vault Drive rule / Drive retention) so recordings/ transcripts follow the schedule.

### *2g. App & bot governance*

1. **Google Chat** → **Chat apps settings.** Set third-party app installation to **restricted / allowlist-only** and populate `Chat App Allowlist – Approved Bots – Enforce` with the approved apps (Requirement #6).

1. Define the request path for adding a new app to the allowlist.

**Rollback (Phase 2):** each setting is reversible in the same console page. Before changing external access, history, retention, or recording storage, record the prior value from `Meet+Chat Baseline – Enterprise – Assess`; to roll back, restore the OU/group setting to that value (for Chat history, re-enable/disable to match baseline; for Vault, disable or delete the retention rule — **note that retention already applied may not be reversible**, so pilot on a non-production OU).

## Phase 3 — Pilot

1. Add a representative set of users to the **Pilot OU** and the special-case groups (`Meet Recorders – Approved`, External Collaboration – Partners`).

1. Have them run real Meet meetings (internal + one external) and use Chat spaces (internal + one allowlisted-external), record a meeting, and share a file in Chat.

1. Collect their experience and any blocked-but-should-be-allowed cases.

**Gate 3:** PASS when pilot users can do their legitimate collaboration and the intended controls visibly enforce (host controls present, external warning shown, recording lands in the Shared Drive, external chat only to allowlisted domains). FAIL → adjust the design and re-configure before expanding.

## Phase 4 — Validate

Run the validation tools and the procedures in **05-testing.md**:

```
python external_retention_policy_validator.py --domain your-domain.com --config design.json --out validati
python meet_chat_policy_health_check.py --domain your-domain.com --out health.json
```

Confirm each gate in the testing guide (controls enforced by OU/group, external scoped to allowlist, recordings in the Shared Drive, retention applied). Name the artifact `Meet+Chat Validation – Pilot – Validate`.

**Gate 4:** PASS only when every test passes for the Pilot OU/group. Any FAIL blocks expansion.

## Phase 5 — Expand

1. Roll the validated policy outward one OU (or wave of OUs) at a time using

`meet_chat_settings_config_helper.py` in `--dry-run` first, then `--apply` under change control:

```
python meet_chat_settings_config_helper.py --config design.json --scope ou:/Sales --dry-run
python meet_chat_settings_config_helper.py --config design.json --scope ou:/Sales --apply
```

1. Re-run the validator and health check for each expanded scope.
2. Keep the rollback (baseline values) ready for each wave; expand only after the prior wave passes.

**Gate 5 (exit):** PASS when all in-scope OUs/groups enforce the design, every wave validated, and the baseline/rollback is documented. The engagement is complete when Meet and Chat are configured for productive, governed collaboration across the enterprise.

# Testing & Validation

Run these after configuring the **Pilot OU/group** (Phase 4) and again for each scope during **Expand** (Phase 5). Each test states the procedure and an explicit **pass/fail** expectation. The validator and health-check tools automate the settings-level checks; the manual tests confirm the lived behavior the API cannot see.

Reference artifacts from Implementation are reused by name below.

## Test 1 — Meet safety & host controls enforce (`Meet Safety Policy – Pilot OU – Configure`)

• **Procedure:** As a pilot user, start a Meet meeting. Confirm **host management** is on (host sees moderation controls: mute all, remove participant, present locks). Have a second internal user join and confirm the host can moderate.

• **Pass:** Host controls are present and functional; join behavior matches the configured "who can join" value.

• **Fail:** Host controls absent, or anyone can join/take over → re-check the OU override in **Meet → Safety settings**; ensure the OU shows the value, not "Inherited".

## Test 2 — External Meet access is scoped (`Meet External Access – Partners Group – Scoped`)

• **Procedure:** As a **general-population** user, attempt to admit an external participant. Then as a `Meet External Access – Partners Group – Scoped` member, do the same.

• **Pass:** General user is blocked (or warned per posture); partner-group user can admit external, and the **external-participant warning banner** shows in both cases.

• **Fail:** General user can freely admit external, or partner user is blocked → re-check the OU/group scoping of the external-access setting.

### Test 3 — Recordings land in the governed Shared Drive (`Recording Storage – Meet Recordings Shared Drive – Configure`)

• **Procedure:** As a `Meet Recorders – Approved` member, record a short pilot meeting. After it finishes, locate the recording and transcript.

• **Pass:** Recording, transcript (and Gemini notes where enabled — *verify in your tenant*) appear in the **Recording Storage Shared Drive**, not in the organizer's My Drive; a non-approved user cannot record.

• **Fail:** Artifact lands in My Drive, or a non-approved user can record → re-check recording enablement scope and the storage-location enforcement (operational or console).

### Test 4 — Chat external policy matches the allowlist (`Chat External Policy – Allowlisted Domains – Enforce`)

• **Procedure:** As a general user, try to start an external chat with (a) an **allowlisted** domain and (b) a **non-allowlisted** domain. Repeat as an `External Collaboration – Partners` member.

• **Pass:** External chat succeeds only to allowlisted domains for permitted users; non-allowlisted and non-permitted attempts are blocked. `external_retention_policy_validator.py` reports the external posture matching `design.json`.

• **Fail:** External chat works to a non-allowlisted domain, or fails to an allowlisted one → re-check external chat setting and the domain allowlist for the OU/group.

### Test 5 — File sharing in Chat follows policy

• **Procedure:** Share a file in an internal space and in an external space.

• **Pass:** Sharing behaves per the configured Chat file-sharing policy (e.g. internal allowed, external restricted).

• **Fail:** File sharing is more permissive than designed → re-check the Chat file-sharing setting.

### Test 6 — Chat history & Vault retention apply (`Chat History – Default On – Enforce`, `Vault Retention – Chat 3yr – Enforce`)

• **Procedure:** Confirm the history default in a new space matches policy. In **Google Vault → Retention → Google Chat**, confirm the rule exists, is scoped correctly, and (in a controlled test) that a message is retained/discoverable per the period.

• **Pass:** History default matches policy; the Vault Chat retention rule is present, correctly scoped, and messages are discoverable in Vault. `meet_chat_policy_health_check.py` flags no history/retention gap for the scope.

• **Fail:** History off where it should be on (or vice-versa), or no retention rule → re-apply in

**Chat** → **Chat history** and Vault; do not expand until retention is confirmed.

### Test 7 — Spaces taxonomy & creation scope (`Chat Spaces Taxonomy – Enterprise – Design`)

- **Procedure:** Attempt to create spaces as permitted and non-permitted users; check that new spaces follow the naming convention and have owners.
- **Pass:** Space creation is limited to permitted users; new spaces conform to naming/ownership.
- **Fail:** Anyone can create spaces or naming is unenforced → re-check space-creation scope and reinforce the taxonomy convention.

### Test 8 — App/bot allowlist enforces (`Chat App Allowlist – Approved Bots – Enforce`)

- **Procedure:** As a pilot user, attempt to install an **approved** Chat app and a **non-approved** one.
- **Pass:** Only approved apps install; non-approved apps are blocked. The assessment/health tool lists installed apps matching the allowlist.
- **Fail:** A non-approved app installs → re-check Chat apps setting is restricted/allowlist-only for the OU.

### Test 9 — Automated settings & health validation

- **Procedure:** Run:

```
python external_retention_policy_validator.py --domain your-domain.com --config design.json --out valida
python meet_chat_policy_health_check.py --domain your-domain.com --out health.json
```

- **Pass:** The validator reports **no drift** between `design.json` and the live settings it can read (external posture, retention presence, recording-storage config where exposed); the health check reports no high-severity findings for the scope.
- **Fail:** Any drift or high-severity finding → remediate the specific control and re-run before expanding.

#### *Overall gate*

The scope **passes** only when Tests 1–9 pass for that OU/group. Record results in `Meet+Chat Validation – Pilot – Validate`. Any failing test blocks expansion to the next wave.

## Troubleshooting

Common failure modes when configuring and validating Google Meet and Chat governance, and how to resolve them. Reference artifacts (e.g. `Meet Safety Policy – Pilot OU – Configure`) are reused

by name.

## Configuration & scoping

### *A setting change doesn't take effect for a user*

• **Symptom:** A pilot user still sees old Meet/Chat behavior after you changed a policy.

• **Cause:** OU/group setting propagation lag; the user isn't in the expected OU/group; or the OU still shows **"Inherited"** rather than the override.

• **Resolution:** Confirm the user's OU under **Directory** → **Users**; confirm the OU shows the explicit value (not "Inherited") in the Meet/Chat settings page; allow propagation time (settings can take up to ~24h to fully apply — *verify current behavior for your tenant*), then re-test.

### *A control isn't available to set*

• **Symptom:** The toggle named in the guide is absent, greyed out, or worded differently.

• **Cause:** The control is edition-gated or the Admin-console UI wording/placement changed since authoring.

• **Resolution:** Confirm the edition includes the control (Requirement #1); search the current Admin console (labels drift — *verify in your tenant*). If the edition lacks it, scope the design to what the edition supports and flag the gap; do not assume an equivalent exists.

### *Configuration-group targeting doesn't override the OU*

• **Symptom:** A `External Collaboration – Partners` group member still gets the restrictive OU policy.

• **Cause:** Chat configuration-group targeting precedence, or the setting is OU-only (not group-targetable).

• **Resolution:** Confirm the setting supports group targeting; verify group membership; check the documented precedence for OU vs. group for that specific setting and adjust the scope accordingly.

## Recording & storage

### *Recordings land in My Drive instead of the Shared Drive*

• **Symptom:** A recorded pilot meeting's file is in the organizer's My Drive, not `Recording Storage – Meet Recordings Shared Drive – Configure`.

• **Cause:** The console does not expose a storage-location control in your edition, or the organizer didn't record into the Shared Drive context.

• **Resolution:** Where a storage-location control exists, set it; otherwise enforce operationally (organizers record from the governed context) and validate location in Test 3. Move stray artifacts into the Shared Drive and apply its retention.

### *A user who shouldn't be able to record can record*

- **Cause:** Recording is enabled at an OU broader than the `Meet Recorders – Approved` scope.
- **Resolution:** Narrow recording enablement to the approved group/OU; re-run Test 3.

## External access & retention

### *External chat works to a non-allowlisted domain*

- **Symptom:** Test 4 fails — external chat reaches a domain not in `Chat External Policy – Allowlisted Domains – Enforce`.
- **Cause:** External chat set to open (not allowlist-only) for the scope, or the allowlist wasn't applied to that OU/group.
- **Resolution:** Set external chat to allowlist-only for the scope, confirm the domain list, re-run the validator; do not expand until it passes.

### *Vault retention rule isn't retaining / discovering messages*

- **Symptom:** A message that should be retained isn't discoverable in Vault.
- **Cause:** Retention rule missing, mis-scoped, Chat history off where retention expects it, or edition lacks Vault.
- **Resolution:** Confirm `Vault Retention – Chat 3yr – Enforce` exists and is scoped to the OU; confirm Chat history is on where the record must be kept; confirm Vault is licensed. **Note:** applied retention can be irreversible — always pilot retention on a non-production OU first.

### *Rolling back external/history/retention has side effects*

- **Cause:** Some governance changes (especially applied Vault retention) are not cleanly reversible.
- **Resolution:** Use the captured `Meet+Chat Baseline – Enterprise – Assess` values to restore reversible settings; for retention, disabling the rule stops future retention but may not undo what was already retained — treat retention changes as forward-only and gate them tightly.

## Automation / API

### `403 insufficient permissions` */ scope errors from a tool*

- **Cause:** The OAuth client or service account lacks the read-only Directory/Reports scopes, or domain-wide delegation isn't authorized for the impersonated admin.
- **Resolution:** Authorize the exact scopes the tool declares in **Security → API controls → Domain-wide delegation**; pass the correct `--subject`; re-run. Grant a write scope only to the config helper and only under change control.

### `429` */ rate-limit / quota errors on large tenants*

- **Symptom:** Assessment or usage report fails part-way with quota/429 errors.
- **Cause:** Admin SDK / Reports API per-minute quota exceeded when enumerating many OUs/groups/days.
- **Resolution:** Re-run (the tools page through results and skip not-yet-available report dates);

reduce `--days`; run during off-peak; if persistent, request a quota increase for the project in the Google Cloud console. The tools are read-only, so re-running is safe.

### *Usage report shows empty or missing parameters*

- **Cause:** Reports data lags ~2–3 days, or the parameter isn't emitted by your edition/version.
- **Resolution:** Widen the look-back window past the lag; treat parameters the tool flags as "verify

for your version" as edition-dependent, not errors. Never infer a value the API didn't return.

### *Health check reports drift after a clean rollout*

- **Symptom:** `meet_chat_policy_health_check.py` later flags a HIGH finding on a scope that previously

passed.

- **Cause:** Real drift — a setting was changed out-of-band, an OU reverted to inherited defaults, or a

new unapproved app was installed.

- **Resolution:** This is the tool working as intended. Investigate the specific finding, restore the

designed value, and re-run the validator + health check. Schedule the health check to catch drift between formal reviews.

# Best Practices

Durable practices for running Google Meet and Chat as a governed collaboration platform after the initial configuration.

## Scope by OU and group deliberately

- Apply each control at the **narrowest scope** that satisfies policy. Keep a restrictive baseline at

the top OU and grant exceptions (external Meet, external chat, recording) to **named configuration groups** rather than loosening whole OUs.

- Prefer **configuration groups** for cross-cutting exceptions (partner-facing teams, approved

recorders) so membership — not OU placement — drives the exception, and membership stays auditable.

- Never leave a high-impact setting "Inherited" by accident. Confirm each OU shows the intended value.

## Keep external collaboration deliberate

- Default external Meet and external Chat to **restrictive**; open only to an **approved domain

allowlist** (`Chat External Policy – Allowlisted Domains – Enforce`) reviewed on a schedule.

• Keep the **external-participant warning** on everywhere so users always know when outsiders are present.

• Re-review the allowlist quarterly with security; remove domains that are no longer needed.

## Govern recordings and transcripts centrally

• Direct all recordings, transcripts, and meeting notes to the governed **Recording Storage Shared Drive**, not personal My Drive, so access and retention apply uniformly.

• Restrict **who can record** to an approved group; recording is a data-creation event with retention and privacy implications.

• Apply retention to the Shared Drive so artifacts age out (or are held) per the records schedule.

## Make Chat eDiscovery-ready

• Set the **history default** to match the records policy and control whether users can change it per space.

• Maintain **Vault retention** rules per data type and OU, and rehearse **legal holds** so eDiscovery works when it is needed, not for the first time during a matter.

• Validate retention on a non-production OU before applying broadly — applied retention can be irreversible.

## Run a real spaces taxonomy

• Enforce a **naming convention** and **ownership** for spaces; every space should have an accountable owner and a clear type (team / project / cross-functional / external).

• Restrict space creation where the edition allows, and periodically prune orphaned or empty spaces.

## Contain third-party app risk

• Keep Chat apps on an **allowlist** with a documented request/approval path; review installed apps against the allowlist regularly.

• Treat a new bot with space-content access as a data-access grant — security-review before allowlisting.

## Operate the controls as living policy

• **Pilot-first, always.** Every change to external access, history, retention, or recording storage goes through the Pilot OU and the validation gates before fleet-wide rollout.

• **Baseline before you change.** Keep a current `Meet+Chat Baseline` so any change has a known rollback value.

• **Automate assessment, not just change.** Run the read-only assessment, usage, health, and validator tools on a schedule to catch **drift** (a setting quietly changed, a new unapproved app, an OU that slipped back to inherited defaults) between formal reviews.

• **Verify vendor-dependent features in your tenant.** Meet/Chat labels, Gemini note-taking, and edition-gated controls change; re-verify after Workspace updates rather than assuming last quarter's behavior.

### Keep automation safe and auditable

• Run tools **read-only or** `--dry-run` **by default**; require explicit `--apply` for any write, under change control.

• Use **least-privilege** OAuth/service-account scopes and store credentials via ADC / a secret store — never embed keys. Log what `--apply` changed for the audit trail.

## Deliverables Summary

Everything included in this solution, on one page.

### Documentation

• **Solution Overview** — the Meet + Chat collaboration model: meeting safety/host controls, recording/transcription and storage, external access, Chat spaces + governance, history/retention, and bot/app policy; the reference model, benefits, limitations, and deployment model.

• **Requirements Guide** — the solution-specific prerequisites (edition/Vault, OU + configuration groups, governed Recording Storage Shared Drive, retention/eDiscovery policy, external-collaboration decision, Chat app allowlist, least-privilege API access), each with how to *verify* and *implement*, layered on the shared Google global requirements.

• **Implementation Guide** — exact, ordered, pilot-first steps following **Assess → Design → Configure → Pilot → Validate → Expand**, with every artifact named and pass/fail gates + rollbacks per phase.

• **Testing & Validation** — nine procedures, each with an explicit pass/fail expectation, plus the automated validator/health run.

• **Troubleshooting** — common Meet/Chat governance failure modes (API-limits/throttling, permission/scope errors, external-policy and retention pitfalls, drift) and their resolutions.

• **Best Practices** — durable practices for scaling collaboration governance.

• **Executive Summary** — outcome, approach, and why the engagement is low-risk.

## The 5 Python tools (Google Workspace Admin SDK / Reports API)

All are **read-only or** `--dry-run` **by default**, use **least-privilege OAuth/ADC** credentials, and contain **no hardcoded secrets or keys**.

1. `meet_chat_settings_assessment.py` — *Assessment / discovery (read-only).* Baselines the tenant:

OU tree and configuration groups via the Admin SDK, plus a manual-check worksheet for the UI-only Meet/Chat safety, external, history, recording, and Vault settings. Produces `Meet+Chat Baseline – Enterprise – Assess`.

1. `meet_chat_settings_config_helper.py` — *Configuration / orchestration helper (dry-run default).*

Turns a reviewed `design.json` into a named, ordered change plan; applies the API-supported changes (scoping-group membership) only with `--apply`, and emits an operator run-book for console-only controls (never faked). Used to configure the Pilot OU and expand wave by wave.

1. `external_retention_policy_validator.py` — *Validation / report generator (read-only).* Compares

the external-access and Chat retention design against observed values and reports **drift** with a pass/fail per control; backs the `Chat External Policy – Allowlisted Domains – Enforce` and `Vault Retention – Chat 3yr – Enforce` gates.

1. `meet_chat_usage_report.py` — *Usage / adoption report (Reports API, read-only).* Summarizes

Meet and Chat activity and external exposure over a look-back window into a CSV, to size adoption and prioritize the external-access design.

1. `meet_chat_policy_health_check.py` — *Policy / health / drift check (read-only).* Scores the

governance posture against best-practice checks (external scoped, recording governed, history/ retention present, app allowlist, not-inherited) and flags findings by severity; ideal on a schedule to catch drift between reviews.

# Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

### external_retention_policy_validator.py

### meet_chat_policy_health_check.py

### meet_chat_settings_assessment.py

### meet_chat_settings_config_helper.py

### meet_chat_usage_report.py